

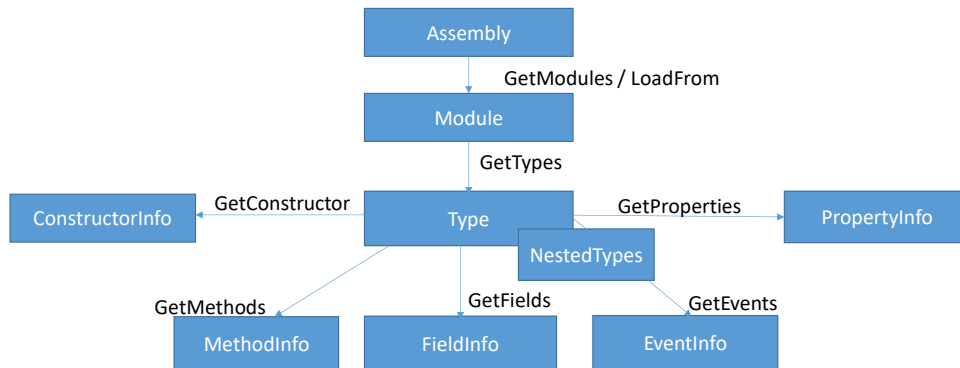


Reflection

Was ist Reflection?

- Zur Laufzeit Ermittlungen von:
 - Klassen
 - Methoden
 - Fields
 - Properties
 - Events
 - Constructor Info
 - NestedTypes
 - Namespaces
 - Interfaces

Grafische Darstellung



Use Cases

- Dynamisches Laden von DLLs
 - Plugin Pattern
- Attribute
- Code Generatoren
- Dynamische Analyse von Objekten

Plugin Pattern

- Interface bildet Schnittstelle zwischen Application und Plugin-Pattern
- Plugin-DLLs werden von der Application zur Laufzeit eingelesen
- Der Plugin Client muss dynamisch prüfen was der Inhalt des Plugins ist

Plugins

- Plugins erben von einem Interface (IPlugin)
 - Wird von PluginBase vorgegeben
- Enthalten u.a. Funktionen, welche von dem Client ausgeführt werden können
- Abhängigkeit zu PluginBase
 - IPlugin
 - ReflectionVisible
 - ...

PluginBase

- Definiert alles, was vom PluginClient und den Plugins selbst verwendet werden soll
- Abhängigkeit vom PluginClient und von allen Plugins
- IPlugin
 - Interface, welches einen gemeinsamen Typen für Client und Plugins bereitstellt
 - Kann beliebige Member enthalten; oft einige beschreibende Member
- ReflectionVisible
 - Attribut, um Member einzublenden, welche beim PluginClient angezeigt werden sollen

PluginClient

- Beliebige Anwendung
 - Konsole, WinForms, WPF, ASP.NET Core, ...
- Lädt Plugins über Reflection
 - Assembly.LoadFrom
 - Entnimmt die Inhalte der Plugins
 - GetParameters()
 - GetMethods()
 - Get...()

Attribute

- Mithilfe von Attributen können Member modifiziert werden
- Attribute können auf bestimmte Typen eingeschränkt werden
- Attribute können nur über Reflection verarbeitet werden
- Attribute können Daten enthalten
 - Werden über den Konstruktor empfangen



DllImport